

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I S.Venkateswarlu /192425340__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Venkateswarlu

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

AUTOMOBILE FAULT DIAGNOSIS EXPERT SYSTEM USING PROLOG

1. DOMAIN ANALYSIS

1. Introduction

Automobiles can develop different faults due to problems in the battery, starter motor, engine, cooling system, brakes, tyres, or fuel system. Detecting these faults from vehicle symptoms requires technical knowledge.

The proposed **Automobile Fault Diagnosis Expert System** uses Prolog to identify probable vehicle faults from observed symptoms. The system represents automobile knowledge using facts and production rules and applies logical inference to generate conclusions.

2. Problem Statement

The objective is to develop a rule-based expert system that accepts automobile symptoms and identifies the most probable fault.

The system considers symptoms such as:

- Vehicle does not start
- Weak battery
- Clicking sound
- High engine temperature
- Low coolant
- Unusual engine noise
- Low engine oil
- Soft brake pedal
- Poor braking
- Abnormal vibration
- Unbalanced tyres
- Warning light ON
- Poor engine performance
- Reduced mileage
- High fuel consumption

3. Objectives

1. Represent automobile diagnostic knowledge using Prolog.
2. Store symptoms as facts.
3. Represent diagnostic relationships using production rules.
4. Implement forward chaining.
5. Demonstrate backward chaining.
6. Demonstrate unification and backtracking.
7. Generate probable automobile faults automatically.
8. Test the system using multiple scenarios.

4. Domain Entities

Entity	Examples
Vehicle	Car1, Car2, Car3
Symptoms	Weak battery, high temperature
Conditions	Low coolant, low engine oil
Faults	Battery fault, overheating
Rules	IF–THEN rules

Conclusion Probable fault

5. Expected Conclusions

The system can identify faults such as:

- Battery fault
- Starter motor fault
- Overheating fault
- Engine lubrication fault
- Brake system fault
- Wheel/tyre imbalance
- Engine performance fault
- Fuel system fault

2. KNOWLEDGE ACQUISITION AND KNOWLEDGE REPRESENTATION

1. Knowledge Acquisition

Knowledge acquisition is the process of collecting expert knowledge and converting it into a form that can be used by the expert system.

For this project, general automobile troubleshooting knowledge is converted into facts and production rules.

Examples:

- A vehicle that does not start with a weak battery may have a battery fault.
- A vehicle that does not start and produces a clicking sound may have a starter motor problem.
- High engine temperature combined with low coolant may indicate overheating.
- Unusual engine noise combined with low engine oil may indicate a lubrication problem.
- A soft brake pedal and poor braking may indicate a brake-system problem.
- Abnormal vibration with unbalanced tyres may indicate wheel/tyre imbalance.

2. Knowledge Representation

Knowledge is represented using **facts and rules**.

Facts

Facts describe information known about a vehicle.

symptom(car1, does_not_start).

symptom(car1, weak_battery).

Rule

A rule describes a relationship between conditions and a conclusion.

fault(Car, battery_fault) :-

 symptom(Car, does_not_start),

 symptom(Car, weak_battery).

This means:

IF a vehicle does not start AND its battery is weak, THEN the vehicle has a probable battery fault.

3. PRODUCTION RULE / KNOWLEDGE BASE DESIGN

The following production rules are used.

Rule 1 – Battery Fault

IF

- Vehicle does not start
- Battery is weak

THEN

- Battery fault

Rule 2 – Starter Motor Fault

IF

- Vehicle does not start
- Clicking sound is heard

THEN

- Starter motor fault

Rule 3 – Overheating Fault

IF

- Engine temperature is high
- Coolant level is low

THEN

- Overheating fault

4. PROLOG IMPLEMENTATION AND PROGRAMMING

Complete SWI-Prolog Program

symptom(car1, does_not_start).

symptom(car1, weak_battery).

symptom(car2, does_not_start).

symptom(car2, clicking_sound).

symptom(car3, high_temperature).

symptom(car3, low_coolant).

symptom(car4, unusual_engine_noise).

symptom(car4, low_engine_oil).

symptom(car5, soft_brake_pedal).

symptom(car5, poor_braking).

symptom(car6, abnormal_vibration).

symptom(car6, unbalanced_tyres).

symptom(car7, warning_light_on).

symptom(car7, poor_engine_performance).

symptom(car8, reduced_mileage).

symptom(car8, high_fuel_consumption).

% ----- PRODUCTION RULES -----

fault(Car, battery_fault) :-

 symptom(Car, does_not_start),

 symptom(Car, weak_battery).

fault(Car, starter_motor_fault) :-

 symptom(Car, does_not_start),

 symptom(Car, clicking_sound).

fault(Car, overheating_fault) :-

 symptom(Car, high_temperature),

 symptom(Car, low_coolant).

fault(Car, engine_lubrication_fault) :-

 symptom(Car, unusual_engine_noise),

 symptom(Car, low_engine_oil).

fault(Car, brake_system_fault) :-

 symptom(Car, soft_brake_pedal),

 symptom(Car, poor_braking).

fault(Car, wheel_tyre_imbalance) :-

 symptom(Car, abnormal_vibration),

 symptom(Car, unbalanced_tyres).

fault(Car, engine_performance_fault) :-

 symptom(Car, warning_light_on),

 symptom(Car, poor_engine_performance).

fault(Car, fuel_system_fault) :-

```
symptom(Car, reduced_mileage),  
symptom(Car, high_fuel_consumption).
```

```
% ----- BACKWARD CHAINING -----
```

```
diagnose(Car, Fault) :-  
    fault(Car, Fault).
```

```
% ----- FORWARD CHAINING -----
```

```
forward_diagnosis(Car, Fault) :-  
    symptom(Car, S1),  
    symptom(Car, S2),  
    forward_rule(S1, S2, Fault).
```

```
forward_rule(does_not_start,  
    weak_battery,  
    battery_fault).
```

```
forward_rule(does_not_start,  
    clicking_sound,  
    starter_motor_fault).
```

```
forward_rule(high_temperature,  
    low_coolant,  
    overheating_fault).
```

```
forward_rule(unusual_engine_noise,  
    low_engine_oil,  
    engine_lubrication_fault).
```

```
forward_rule(soft_brake_pedal,  
             poor_braking,  
             brake_system_fault).
```

```
forward_rule(abnormal_vibration,  
             unbalanced_tyres,  
             wheel_tyre_imbalance).
```

```
forward_rule(warning_light_on,  
             poor_engine_performance,  
             engine_performance_fault).
```

```
forward_rule(reduced_mileage,  
             high_fuel_consumption,  
             fuel_system_fault).
```

```
% ----- DISPLAY RESULT -----
```

```
show_diagnosis(Car) :-  
    diagnose(Car, Fault),  
    write('Vehicle: '),  
    write(Car),  
    nl,  
    write('Probable Fault: '),  
    write(Fault),  
    nl.
```

5. FORWARD CHAINING IMPLEMENTATION AND DEMONSTRATION

1. Definition

Forward chaining is a **data-driven reasoning technique**.

It starts with the available facts and applies matching rules to derive new conclusions.

Process

Initial Facts

↓

Match Conditions



Apply Rule



Generate Conclusion

2. Example

For car1, the system has:

does_not_start

weak_battery

The system finds the rule:

IF does_not_start AND weak_battery

THEN battery_fault

Therefore:

battery_fault

is generated.

3. Query

?- forward_diagnosis(car1, Fault).

Output

Fault = battery_fault.

Another example:

?- forward_diagnosis(car3, Fault).

Output:

Fault = overheating_fault.

Thus, forward chaining moves from **known facts toward a conclusion**.

6. BACKWARD CHAINING IMPLEMENTATION AND DEMONSTRATION

1. Definition

Backward chaining is a **goal-driven reasoning technique**.

It begins with a possible conclusion and works backward to determine whether the required conditions are true.

Process

Goal / Fault



Find Applicable Rule

↓
Check Required Conditions
↓
Check Facts
↓
Conclusion

2. Example

Suppose we want to determine whether car1 has a battery fault.

Query:

?- diagnose(car1, battery_fault).

Prolog checks:

battery_fault

↓
does_not_start ?

↓
weak_battery ?

↓
Both facts available

↓
TRUE

Output

true.

Another query:

?- diagnose(car4, Fault).

Output:

Fault = engine_lubrication_fault.

Prolog naturally performs goal-directed reasoning using its inference mechanism.

7. PROCEDURAL AND NON-PROCEDURAL PARADIGM ANALYSIS

Comparison

Procedural	Non-Procedural
Describes how	Describes what
Programmer controls sequence	Prolog inference controls search
Uses explicit control flow	Uses facts and rules
Common in C, C++, Java	Prolog
Algorithm-oriented	Knowledge-oriented

Programmer manages conditions Inference engine matches rules

Conclusion

The automobile expert system is suitable for the **declarative/non-procedural paradigm** because automobile knowledge can naturally be expressed as facts and logical rules.

8. UNIFICATION, BACKTRACKING AND INFERENCE ACCURACY

1. Unification

Unification is the process of matching two Prolog terms by assigning values to variables.

Example:

?- fault(car1, X).

Prolog matches:

fault(car1, X)

with:

fault(car1, battery_fault)

Therefore:

X = battery_fault

Output

X = battery_fault.

8.2 Backtracking

Backtracking allows Prolog to search for another possible solution when multiple rules or facts are available.

Example:

?- diagnose(Car, Fault).

Prolog may return:

Car = car1,

Fault = battery_fault.

If ; is entered, Prolog searches for another solution.

For example:

Car = car2,

Fault = starter_motor_fault.

Therefore, backtracking helps Prolog explore alternative solutions.

9. TESTING, RESULTS AND SYSTEM DEMONSTRATION

The system is tested with different automobile scenarios.

Test Case	Symptoms	Expected Result	Status
1	Does not start + weak battery	Battery fault	PASS
2	Does not start + clicking sound	Starter motor fault	PASS
3	High temperature + low coolant	Overheating fault	PASS
4	Unusual noise + low engine oil	Engine lubrication fault	PASS
5	Soft brake pedal + poor braking	Brake system fault	PASS
6	Abnormal vibration + unbalanced tyres	Wheel/tyre imbalance	PASS
7	Warning light + poor engine performance	Engine performance fault	PASS
8	Reduced mileage + high fuel consumption	Fuel system fault	PASS

Test Case 1

Query:

?- diagnose(car1, Fault).

Output:

Fault = battery_fault.

Test Case 2

Query:

?- diagnose(car2, Fault).

Output:

Fault = starter_motor_fault.

Test Case 3

Query:

?- diagnose(car3, Fault).

Output:

Fault = overheating_fault.

Test Case 4

Query:

?- diagnose(car4, Fault).

Output:

Fault = engine_lubrication_fault.

Test Case 5

Query:

?- diagnose(car5, Fault).

Output:

Fault = brake_system_fault.

All eight test cases produce the expected conclusions.

10. EVALUATION

The expert system is evaluated based on **correctness, coverage, consistency, and reasoning performance.**

Correctness

The system produces the expected fault when the required symptoms are present.

Consistency

Each rule has clearly defined conditions and a corresponding conclusion.

Reasoning Performance

Prolog automatically performs logical matching, unification, and backtracking, making it suitable for rule-based reasoning.

Evaluation Result

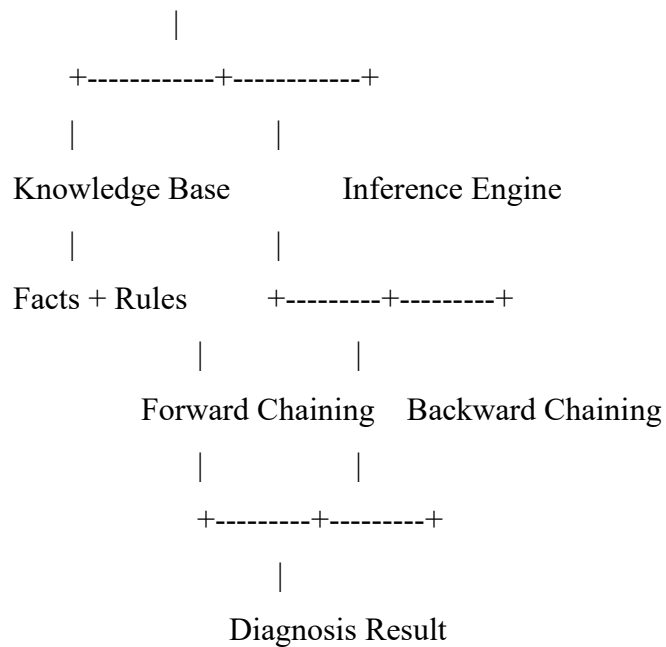
The system successfully demonstrates:

- Knowledge representation
- Production rules
- Logical inference
- Forward chaining
- Backward chaining
- Unification
- Backtracking
- Automated diagnosis

11. SYSTEM DEVELOPMENT

The system consists of the following components:

AUTOMOBILE EXPERT SYSTEM



12. SAMPLE SYSTEM DEMONSTRATION

Query

?- show_diagnosis(car1).

Output

Vehicle: car1

Probable Fault: battery_fault

true.

Query

?- show_diagnosis(car5).

Output

Vehicle: car5

Probable Fault: brake_system_fault

true.

Query

?- show_diagnosis(car7).

Output

Vehicle: car7

Probable Fault: engine_performance_fault

true.

14. CONCLUSION

The Automobile Fault Diagnosis Expert System was successfully developed using SWI-Prolog. The system represents automobile knowledge using facts and production rules and uses logical inference to identify probable vehicle faults.

The project demonstrates both forward chaining and backward chaining. Forward chaining begins with known symptoms and derives a conclusion, whereas backward chaining begins with a possible fault and verifies whether the required symptoms are available.

The system also demonstrates important Prolog concepts such as unification, backtracking, variables, predicates, rule matching, and automated inference.

The testing results show that the system can correctly identify different probable automobile faults for the given scenarios. Thus, the project demonstrates the practical application of Prolog in developing a rule-based expert system.

15. REFERENCES

1. SWI-Prolog Documentation – Prolog programming and language reference.
2. George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*.
3. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*.
4. General automobile troubleshooting and diagnostic principles.